

0.87.6. Agrandir une lettre : `texteGrand` — aller à la ligne, `texteGrandS`

`texteGrandS(0, 0, "BONJOUR", 2)` : comme `texteGrand`, mais les lettres qui ne tiennent plus dans la largeur passent à la ligne, comme `textS`.

1. Ce qu'il faut comprendre

Avec `texteGrand`, un mot trop long sort de l'écran par la droite. « BONJOUR » en taille 2 (3 cases par lettre) demande $7 \times 3 = 21$ cases de large ; l'écran n'en a que 20 : le R disparaît.

Ce qui est nouveau ici : `texteGrandS`, le `S` de `textS` (le 0.11). Même réglages que `texteGrand`, mais les lettres qui ne tiennent plus **passent à la ligne** : elles reprennent en colonne 0, une rangée de lettres plus bas (3 cases, à la taille 2).

Le découpage : à la taille 2, une lettre prend 3 cases ; une ligne en tient $20 \div 3 = 6$ lettres (18 cases). « BONJOU » sur la première, « R » sur la deuxième.

Trop haut, c'est une erreur : l'écran a 18 cases de haut. Si le texte en demande plus (« BONJOUR » en taille 6 : 4 lignes de 7 cases, 28), le compilateur refuse, et dit combien il en faudrait.

Tout s'écrit en clair, même la place : le découpage en lignes se fait avant le jeu.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteGrandS>` → [0.87.5. La fonction `texteGrandS\(\)` — un grand texte qui va à la ligne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1704 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          61 o ← la console, à l'allumage — la mise en route de la console
texte_grand_0()     146 o ← main() — ta fonction
texte_grand_1()     146 o ← main() — ta fonction
main()              26 o ← démarrage — ta fonction
VBlank              8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
AttendreVBlank      16 o ← démarrage — attend que l'écran soit libre
AdresseCase         10 o ← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles        18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage — vide les deux cartes de l'écran
Tuiles              1168 o ← CopierTuiles — la police entière, et 29 tuiles dessinées
GRAND_TABLE_0_0_0   18 o ← poser() — une table gravée
GRAND_TABLE_0_1_1   18 o ← poser() — une table gravée
GRAND_TABLE_0_2_2   18 o ← poser() — une table gravée
GRAND_TABLE_1_0_3    3 o ← poser() — une table gravée
GRAND_TABLE_1_1_4    3 o ← poser() — une table gravée
GRAND_TABLE_1_2_5    3 o ← poser() — une table gravée
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser() × 6 182 o sur place, dans texte_grand_0(), texte_grand_1() + AdresseCase (10 o),
GRAND_TABLE_0_0_0 (18 o), GRAND_TABLE_0_1_1 (18 o), GRAND_TABLE_0_2_2 (18 o), GRAND_TABLE_1_0_3 (3 o),
GRAND_TABLE_1_1_4 (3 o), GRAND_TABLE_1_2_5 (3 o)

image() × 1 3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

texte_grand_0() × 1 3 o sur place, dans main()

texte_grand_1() × 1 3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

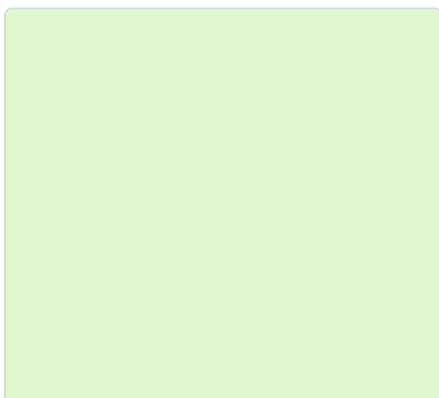
```
#include <texteGrandS> // écrit un texte agrandi qui passe à la ligne
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

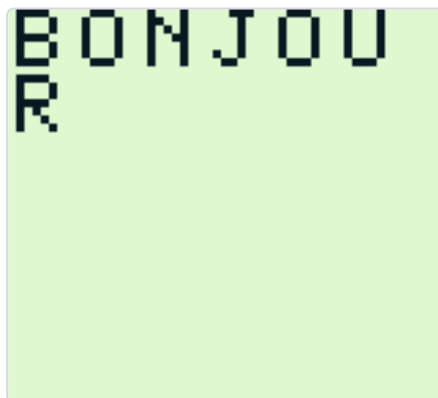
```
int main() {  
    // Comme texteGrand, mais qui VA À LA LIGNE :  
    //  
    // texteGrandS(0, 0, "BONJOUR", 2);  
    // la taille 2 : 3 cases par lettre ; une ligne de 20 cases en tient 6.  
    //  
    // ligne 1, cases 0 à 2 : B O N J O U (6 lettres × 3 = 18 cases)  
    // ligne 2, cases 3 à 5 : R (repart en colonne 0)  
    texteGrandS(0, 0, "BONJOUR", 2);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

BONJOU en grandes lettres sur la première ligne, et le R en dessous, à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ BONJOU sur la première ligne (cases 0 à 2)

✓ le R est passé à la ligne, en colonne 0

✓ rien après le R

Ensuite : **0.87.7. La fonction `couleurTexte()` — la couleur des lettres** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++